

Shuhan_Zhang_R_Code_Sample_Firm_Investment_Oil_Shocks

```
1 # =====
2 # R CODE SAMPLE
3 # Firm-Level Investment Responses to Oil Price Shocks
4 # =====
5 #
6 # Purpose
7 # -----
8 # This script demonstrates an end-to-end empirical workflow in R. It:
9 #   1. imports and validates a firm-year panel;
10 #   2. constructs treatment variables and consecutive-year lags;
11 #   3. aligns monthly structural oil shocks with firm-specific fiscal years;
12 #   4. estimates two-way fixed-effects panel models with multiway clustered
13 #       standard errors;
14 #   5. evaluates dynamic cumulative effects; and
15 #   6. conducts selected robustness checks.
16 #
17 # Research question
18 # -----
19 # Do positive oil-price shocks reduce firm-level capital investment, and are
20 # these effects larger for firms operating in energy-intensive industries?
21 #
22 # Data note
23 # -----
24 # The firm-level data are proprietary and therefore are not distributed with
25 # this code sample. Relative paths below illustrate the intended project
26 # structure. Variable definitions are summarized where they first appear.
27 #
28 # Reproducibility note
29 # -----
30 # This script assumes one observation per firm-year. All lagged variables are
31 # constructed only when fiscal years are consecutive, which prevents an
32 # unbalanced panel from incorrectly treating a multi-year gap as a one-year lag.
33 # =====
34
35 # -----
36 # 1. Package setup and project paths
37 # -----
38
39 required_packages <- c(
40   "car",
41   "data.table",
42   "dplyr",
43   "fixest",
44   "here",
45   "knitr",
46   "lubridate",
47   "readr",
48   "readxl",
49   "splines",
50   "tidyr"
51 )
52
53 missing_packages <- required_packages[
54   !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
55 ]
56
57 if (length(missing_packages) > 0L) {
58   stop(
```

```

59 "Install the following packages before running this script: ",
60 paste(missing_packages, collapse = ", ")
61 )
62 }
63
64 library(car)
65 library(data.table)
66 library(dplyr)
67 library(fixest)
68 library(here)
69 library(knitr)
70 library(lubridate)
71 library(readr)
72 library(readxl)
73 library(splines)
74 library(tidyr)
75
76 # Use relative paths so the analysis is portable across computers.
77 paths <- list(
78   panel = here("data", "processed", "thesis_model_df.csv"),
79   controls = here("data", "processed", "panel_patch_controls.csv"),
80   supply_shocks = here("data", "raw", "BH2_supply_shocks.xlsx"),
81   demand_shocks = here("data", "raw", "BH2_demand_shocks.xlsx"),
82   tables = here("output", "tables")
83 )
84
85 dir.create(paths$tables, recursive = TRUE, showWarnings = FALSE)
86
87 # -----
88 # 2. Reusable helper functions
89 # -----
90
91 # Winsorize a numeric vector by replacing values below and above the selected
92 # quantiles. This limits the influence of extreme accounting observations while
93 # preserving the sample size.
94 winsorize_vec <- function(x, lower = 0.01, upper = 0.99) {
95   if (!is.numeric(x)) {
96     stop("winsorize_vec() requires a numeric vector.")
97   }
98   if (lower < 0 || upper > 1 || lower >= upper) {
99     stop("Choose quantile bounds satisfying 0 <= lower < upper <= 1.")
100   }
101
102   bounds <- stats::quantile(
103     x,
104     probs = c(lower, upper),
105     na.rm = TRUE,
106     names = FALSE
107   )
108
109   pmin(pmax(x, bounds[1]), bounds[2])
110 }
111
112 # Construct a one-period lag only when the current and previous observations
113 # belong to consecutive years. This is essential for an unbalanced panel.
114 consecutive_lag <- function(x, year) {
115   lagged_x <- dplyr::lag(x)
116   lagged_year <- dplyr::lag(year)
117
118   dplyr::if_else(
119     !is.na(lagged_year) & year == lagged_year + 1L,
120     lagged_x,

```

```

121 NA_real_
122 )
123 }
124
125 # Calculate the cumulative effect of a current and lagged coefficient using the
126 # full variance-covariance matrix. The covariance term is necessary because the
127 # two estimates are obtained from the same regression.
128 cumulative_effect <- function(model, current_term, lagged_term) {
129   coefficients <- stats::coef(model)
130   variance_matrix <- stats::vcov(model)
131
132   required_terms <- c(current_term, lagged_term)
133   if (!all(required_terms %in% names(coefficients))) {
134     stop("The requested coefficient names are not present in the model.")
135   }
136
137   estimate <- coefficients[current_term] + coefficients[lagged_term]
138   standard_error <- sqrt(
139     variance_matrix[current_term, current_term] +
140     variance_matrix[lagged_term, lagged_term] +
141     2 * variance_matrix[current_term, lagged_term]
142   )
143
144   z_statistic <- estimate / standard_error
145   p_value <- 2 * stats::pnorm(abs(z_statistic), lower.tail = FALSE)
146
147   tibble(
148     estimate = unname(estimate),
149     standard_error = unname(standard_error),
150     statistic = unname(z_statistic),
151     p_value = unname(p_value)
152   )
153 }
154
155 # Stop early when a required variable is missing. Explicit validation makes
156 # later modeling errors easier to diagnose.
157 validate_columns <- function(data, required_columns, data_name) {
158   missing_columns <- setdiff(required_columns, names(data))
159
160   if (length(missing_columns) > 0L) {
161     stop(
162       data_name,
163       " is missing required columns: ",
164       paste(missing_columns, collapse = ", ")
165     )
166   }
167 }
168
169 # -----
170 # 3. Import, merge, and validate the firm-year data
171 # -----
172
173 # The Python data-preparation step creates two processed files:
174 # 1. thesis_model_df.csv contains the main regression-ready panel; and
175 # 2. panel_patch_controls.csv contains supplemental balance-sheet controls
176 #    and fiscal year-end dates used for fiscal-year shock alignment.
177 #
178 # Some earlier versions of the processed panel did not include datadate,
179 # cash_at, or tangibility directly. The merge below is therefore written to be
180 # robust: it validates core columns in the main panel first, then attaches the
181 # supplemental controls from the patch file.
182 firm_panel <- read_csv(paths$panel, show_col_types = FALSE)

```

```

183 control_patch <- read_csv(paths$controls, show_col_types = FALSE)
184
185 validate_columns(
186   firm_panel,
187   c(
188     "gvkey", "fyear", "energy_intensity", "oil_shock",
189     "inv_at", "inv_at_w", "inv_ppent_w", "ln_at", "lev", "lev_w",
190     "prof", "prof_w", "sales_growth", "sales_growth_w"
191   ),
192   "firm_panel"
193 )
194
195 validate_columns(
196   control_patch,
197   c("gvkey", "fyear", "datadate", "cash_at", "tangibility"),
198   "control_patch"
199 )
200
201 # Harmonize join keys before combining the processed data sources.
202 firm_panel <- firm_panel %>%
203   mutate(
204     gvkey = as.character(gvkey),
205     fyear = as.integer(fyear)
206   )
207
208 # If the main panel already contains datadate, keep it; otherwise create a
209 # placeholder that will be filled from the patch file after the join.
210 if ("datadate" %in% names(firm_panel)) {
211   firm_panel <- firm_panel %>% mutate(datadate = as.Date(datadate))
212 } else {
213   firm_panel$datadate <- as.Date(NA)
214 }
215
216 control_patch <- control_patch %>%
217   mutate(
218     gvkey = as.character(gvkey),
219     fyear = as.integer(fyear),
220     datadate_patch = as.Date(datadate)
221   )
222
223 # Each input should have one record per firm-year before the merge. This avoids
224 # accidental many-to-many joins that would duplicate observations.
225 if (anyDuplicated(firm_panel[c("gvkey", "fyear")]) > 0L) {
226   stop("The main panel contains duplicate firm-year observations.")
227 }
228
229 if (anyDuplicated(control_patch[c("gvkey", "fyear")]) > 0L) {
230   stop("The control patch contains duplicate firm-year observations.")
231 }
232
233 control_patch_for_join <- control_patch %>%
234   select(
235     gvkey,
236     fyear,
237     datadate_patch,
238     cash_at,
239     tangibility,
240     any_of(c("cogs_share", "sga_intensity", "op_margin"))
241   )
242
243 firm_panel <- firm_panel %>%
244   left_join(control_patch_for_join, by = c("gvkey", "fyear")) %>%

```

```

245 mutate(datadate = coalesce(datadate, datadate_patch)) %>%
246 select(-datadate_patch)
247
248 validate_columns(
249   firm_panel,
250   c("datadate", "cash_at", "tangibility"),
251   "merged_firm_panel"
252 )
253
254 # Each record must represent a unique firm-year. Duplicate records would make
255 # both lag construction and fixed-effects estimation ambiguous.
256 if (anyDuplicated(firm_panel[c("gvkey", "fyear")]) > 0L) {
257   stop("The merged panel contains duplicate firm-year observations.")
258 }
259
260 if (any(is.na(firm_panel$gvkey)) || any(is.na(firm_panel$fyear))) {
261   stop("Firm identifiers and fiscal years must not be missing.")
262 }
263
264 if (all(is.na(firm_panel$datadate))) {
265   stop("Fiscal year-end dates are missing, so fiscal-year shock alignment cannot be performed.")
266 }
267
268 message(
269   "Panel coverage: ",
270   min(firm_panel$fyear, na.rm = TRUE),
271   "-",
272   max(firm_panel$fyear, na.rm = TRUE),
273   "; firms: ",
274   n_distinct(firm_panel$gvkey),
275   "; observations: ",
276   nrow(firm_panel)
277 )
278
279 # -----
280 # 4. Construct treatment variables, controls, and valid lags
281 # -----
282
283 # Define high-energy industries using the 75th percentile of the time-invariant
284 # industry energy-intensity measure.
285 high_energy_cutoff <- quantile(
286   firm_panel$energy_intensity,
287   probs = 0.75,
288   na.rm = TRUE,
289   names = FALSE
290 )
291
292 firm_panel <- firm_panel %>%
293   mutate(
294     high_energy = as.integer(energy_intensity >= high_energy_cutoff),
295
296     # Positive-only shocks allow the analysis to focus on oil-price increases,
297     # which are the theoretically relevant adverse cost shocks.
298     oil_shock_pos = pmax(oil_shock, 0),
299
300     # Winsorize selected accounting ratios at the 1st and 99th percentiles.
301     cash_at_w = winsorize_vec(cash_at),
302     tangibility_w = winsorize_vec(tangibility)
303   ) %>%
304   arrange(gvkey, fyear, datadate) %>%
305   group_by(gvkey) %>%
306   mutate(

```

```

307 oil_shock_pos_l1 = consecutive_lag(oil_shock_pos, fyear),
308 ln_at_l1 = consecutive_lag(ln_at, fyear),
309 lev_w_l1 = consecutive_lag(lev_w, fyear),
310 prof_w_l1 = consecutive_lag(prof_w, fyear),
311 sales_growth_w_l1 = consecutive_lag(sales_growth_w, fyear),
312 cash_at_w_l1 = consecutive_lag(cash_at_w, fyear),
313 tangibility_w_l1 = consecutive_lag(tangibility_w, fyear)
314 ) %>%
315 ungroup() %>%
316 mutate(
317   oil_x_high = oil_shock * high_energy,
318   oil_pos_x_high = oil_shock_pos * high_energy,
319   oil_pos_x_high_l1 = oil_shock_pos_l1 * high_energy
320 )
321
322 # -----
323 # 5. Import and clean monthly structural oil shocks
324 # -----
325
326 # The source spreadsheets contain two descriptive header rows and do not use
327 # analysis-ready column names, so the relevant fields are selected explicitly.
328 supply_raw <- read_excel(paths$supply_shocks, col_names = FALSE)
329 demand_raw <- read_excel(paths$demand_shocks, col_names = FALSE)
330
331 supply_shocks <- supply_raw %>%
332   slice(-(1:2)) %>%
333   select(1, 2) %>%
334   rename(
335     date = ...1,
336     oil_supply_shock = ...2
337   ) %>%
338   mutate(
339     date = as.Date(date),
340     oil_supply_shock = as.numeric(oil_supply_shock)
341   )
342
343 demand_shocks <- demand_raw %>%
344   slice(-(1:2)) %>%
345   select(1, 2, 3, 4) %>%
346   rename(
347     date = ...1,
348     economic_activity_shock = ...2,
349     oil_consumption_demand_shock = ...3,
350     oil_inventory_demand_shock = ...4
351   ) %>%
352   mutate(
353     date = as.Date(date),
354     across(
355       c(
356         economic_activity_shock,
357         oil_consumption_demand_shock,
358         oil_inventory_demand_shock
359       ),
360       as.numeric
361     )
362   )
363
364 monthly_shocks <- supply_shocks %>%
365   left_join(demand_shocks, by = "date") %>%
366   arrange(date)
367
368 if (anyDuplicated(monthly_shocks$date) > 0L) {

```

```

369   stop("The monthly shock data contain duplicate dates.")
370 }
371
372 # -----
373 # 6. Align monthly shocks with firm-specific fiscal years
374 # -----
375
376 # A calendar-year aggregation would misclassify exposure for firms whose fiscal
377 # years do not end in December. Instead, each firm-year is assigned the monthly
378 # shocks observed during its own 12-month reporting window.
379 firm_fiscal_windows <- firm_panel %>%
380   mutate(
381     fiscal_end_month = floor_date(datadate, unit = "month"),
382     fiscal_start_month = fiscal_end_month %m-% months(11)
383   )
384
385 firm_dt <- as.data.table(firm_fiscal_windows)
386 shock_dt <- as.data.table(monthly_shocks)
387
388 # Non-equijoin: match each monthly observation to every firm-year fiscal window
389 # that contains that month.
390 matched_months <- shock_dt[
391   firm_dt,
392   on = .(
393     date >= fiscal_start_month,
394     date <= fiscal_end_month
395   ),
396   allow.cartesian = TRUE,
397   nomatch = 0L
398 ]
399
400 # Confirm that most firm-years are matched to 12 monthly observations. Keeping
401 # this check visible documents the temporal alignment and reveals incomplete
402 # windows near the boundaries of the shock series.
403 match_counts <- matched_months[, .N, by = .(gvkey, fyear)]
404 message(
405   "Fiscal-year monthly matches: median = ",
406   stats::median(match_counts$N),
407   "; range = ",
408   min(match_counts$N),
409   "-",
410   max(match_counts$N)
411 )
412
413 fiscal_shocks <- matched_months[
414   ,
415   .(
416     oil_supply_shock_fy = mean(oil_supply_shock, na.rm = TRUE),
417     economic_activity_shock_fy = mean(
418       economic_activity_shock,
419       na.rm = TRUE
420     ),
421     oil_consumption_demand_shock_fy = mean(
422       oil_consumption_demand_shock,
423       na.rm = TRUE
424     ),
425     oil_inventory_demand_shock_fy = mean(
426       oil_inventory_demand_shock,
427       na.rm = TRUE
428     ),
429     matched_months = .N
430   ),

```

```

431   by = .(gvkey, fyear)
432 ]
433
434 firm_panel <- firm_panel %>%
435   left_join(as_tibble(fiscal_shocks), by = c("gvkey", "fyear")) %>%
436   arrange(gvkey, fyear, datadate) %>%
437   group_by(gvkey) %>%
438   mutate(
439     supply_x_high_fy = oil_supply_shock_fy * high_energy,
440     activity_x_high_fy = economic_activity_shock_fy * high_energy,
441     supply_x_high_fy_l1 = consecutive_lag(supply_x_high_fy, fyear),
442     activity_x_high_fy_l1 = consecutive_lag(activity_x_high_fy, fyear)
443   ) %>%
444   ungroup()
445
446 # -----
447 # 7. Define common analysis samples
448 # -----
449
450 # Using a common sample ensures coefficient changes across specifications are
451 # driven by the model rather than by different missing-value patterns.
452 control_variables <- c(
453   "ln_at_l1",
454   "lev_w_l1",
455   "prof_w_l1",
456   "sales_growth_w_l1",
457   "cash_at_w_l1",
458   "tangibility_w_l1"
459 )
460
461 reduced_form_sample <- firm_panel %>%
462   drop_na(
463     inv_at_w,
464     oil_x_high,
465     oil_pos_x_high,
466     oil_pos_x_high_l1,
467     all_of(control_variables)
468   )
469
470 decomposition_sample <- firm_panel %>%
471   drop_na(
472     inv_at_w,
473     supply_x_high_fy,
474     supply_x_high_fy_l1,
475     activity_x_high_fy,
476     activity_x_high_fy_l1,
477     all_of(control_variables)
478   )
479
480 message(
481   "Reduced-form sample: ", nrow(reduced_form_sample), " observations; ",
482   n_distinct(reduced_form_sample$gvkey), " firms."
483 )
484 message(
485   "Decomposition sample: ", nrow(decomposition_sample), " observations; ",
486   n_distinct(decomposition_sample$gvkey), " firms."
487 )
488
489 # -----
490 # 8. Estimate two-way fixed-effects models
491 # -----
492

```



```

493 # All models include firm and fiscal-year fixed effects. Standard errors are
494 # clustered by both firm and year to allow arbitrary serial correlation within
495 # firms and common shocks within years.
496 model_baseline <- feols(
497   inv_at_w ~ oil_x_high +
498     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
499     cash_at_w_l1 + tangibility_w_l1 |
500     gvkey + fyear,
501   data = reduced_form_sample,
502   vcov = ~ gvkey + fyear
503 )
504
505 model_positive_shock <- feols(
506   inv_at_w ~ oil_pos_x_high +
507     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
508     cash_at_w_l1 + tangibility_w_l1 |
509     gvkey + fyear,
510   data = reduced_form_sample,
511   vcov = ~ gvkey + fyear
512 )
513
514 # The dynamic specification includes current and one-year-lagged exposure. This
515 # captures delayed capital-budget responses and permits a cumulative-effect test.
516 model_dynamic <- feols(
517   inv_at_w ~ oil_pos_x_high + oil_pos_x_high_l1 +
518     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
519     cash_at_w_l1 + tangibility_w_l1 |
520     gvkey + fyear,
521   data = reduced_form_sample,
522   vcov = ~ gvkey + fyear
523 )
524
525 # Structural decomposition separates supply-driven oil shocks from shocks tied
526 # to aggregate economic activity. The distinction helps identify whether higher
527 # oil prices reflect adverse cost pressure or stronger macroeconomic demand.
528 model_supply_shock <- feols(
529   inv_at_w ~ supply_x_high_fy + supply_x_high_fy_l1 +
530     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
531     cash_at_w_l1 + tangibility_w_l1 |
532     gvkey + fyear,
533   data = decomposition_sample,
534   vcov = ~ gvkey + fyear
535 )
536
537 model_activity_shock <- feols(
538   inv_at_w ~ activity_x_high_fy + activity_x_high_fy_l1 +
539     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
540     cash_at_w_l1 + tangibility_w_l1 |
541     gvkey + fyear,
542   data = decomposition_sample,
543   vcov = ~ gvkey + fyear
544 )
545
546 # -----
547 # 9. Evaluate cumulative current-plus-lagged effects
548 # -----
549
550 cumulative_results <- bind_rows(
551   cumulative_effect(
552     model_dynamic,
553     "oil_pos_x_high",
554     "oil_pos_x_high_l1"

```

```

555   ) %>%
556     mutate(model = "Positive oil-price shock"),
557
558     cumulative_effect(
559       model_supply_shock,
560       "supply_x_high_fy",
561       "supply_x_high_fy_l1"
562     ) %>%
563     mutate(model = "Structural oil-supply shock"),
564
565     cumulative_effect(
566       model_activity_shock,
567       "activity_x_high_fy",
568       "activity_x_high_fy_l1"
569     ) %>%
570     mutate(model = "Economic-activity shock")
571 ) %>%
572   select(model, everything())
573
574 print(cumulative_results)
575
576 # Cross-check the manually calculated variance of the coefficient sum using a
577 # formal linear hypothesis test.
578 print(
579   linearHypothesis(
580     model_dynamic,
581     "oil_pos_x_high + oil_pos_x_high_l1 = 0",
582     vcov. = vcov(model_dynamic)
583   )
584 )
585
586 # -----
587 # 10. Selected robustness checks
588 # -----
589
590 # Robustness check 1: use capital expenditure divided by beginning property,
591 # plant, and equipment as an alternative investment measure.
592 alt_outcome_sample <- firm_panel %>%
593   drop_na(
594     inv_ppent_w,
595     oil_pos_x_high,
596     oil_pos_x_high_l1,
597     all_of(control_variables)
598   )
599
600 model_alt_outcome <- feols(
601   inv_ppent_w ~ oil_pos_x_high + oil_pos_x_high_l1 +
602     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
603     cash_at_w_l1 + tangibility_w_l1 |
604     gvkey + fyear,
605   data = alt_outcome_sample,
606   vcov = ~ gvkey + fyear
607 )
608
609 # Robustness check 2: replace the high-energy indicator with a flexible natural
610 # spline in continuous energy intensity. This allows heterogeneous effects to
611 # vary nonlinearly rather than imposing a single threshold.
612 spline_basis <- ns(firm_panel$energy_intensity, df = 3)
613
614 spline_panel <- firm_panel %>%
615   mutate(
616     spline_1 = spline_basis[, 1],

```

```

617 spline_2 = spline_basis[, 2],
618 spline_3 = spline_basis[, 3],
619 oil_pos_x_spline_1 = oil_shock_pos * spline_1,
620 oil_pos_x_spline_2 = oil_shock_pos * spline_2,
621 oil_pos_x_spline_3 = oil_shock_pos * spline_3
622 ) %>%
623 drop_na(
624   inv_at_w,
625   oil_pos_x_spline_1,
626   oil_pos_x_spline_2,
627   oil_pos_x_spline_3,
628   all_of(control_variables)
629 )
630
631 model_spline <- feols(
632   inv_at_w ~ oil_pos_x_spline_1 + oil_pos_x_spline_2 +
633     oil_pos_x_spline_3 +
634     ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
635     cash_at_w_l1 + tangibility_w_l1 |
636     gvkey + fyear,
637   data = spline_panel,
638   vcov = ~ gvkey + fyear
639 )
640
641 # Jointly test whether the three nonlinear interaction terms are zero.
642 print(wald(model_spline, keep = "oil_pos_x_spline"))
643
644 # -----
645 # 11. Export presentation-ready results
646 # -----
647
648 variable_labels <- c(
649   "oil_x_high" = "Oil shock x high energy intensity",
650   "oil_pos_x_high" = "Positive oil shock x high energy intensity",
651   "oil_pos_x_high_l1" = "Lagged positive oil shock x high energy intensity",
652   "supply_x_high_fy" = "Supply shock x high energy intensity",
653   "supply_x_high_fy_l1" = "Lagged supply shock x high energy intensity",
654   "activity_x_high_fy" = "Activity shock x high energy intensity",
655   "activity_x_high_fy_l1" = "Lagged activity shock x high energy intensity",
656   "ln_at_l1" = "Firm size",
657   "lev_w_l1" = "Leverage",
658   "prof_w_l1" = "Profitability",
659   "sales_growth_w_l1" = "Sales growth",
660   "cash_at_w_l1" = "Cash holdings",
661   "tangibility_w_l1" = "Tangibility"
662 )
663
664 etable(
665   list(
666     "Baseline" = model_baseline,
667     "Positive shock" = model_positive_shock,
668     "Dynamic positive shock" = model_dynamic
669   ),
670   dict = variable_labels,
671   tex = TRUE,
672   replace = TRUE,
673   digits = 4,
674   file = file.path(paths$tables, "reduced_form_models.tex")
675 )
676
677 etable(
678   list(

```

```
679     "Supply shock" = model_supply_shock,  
680     "Activity shock" = model_activity_shock,  
681     "Alternative outcome" = model_alt_outcome  
682   ),  
683   dict = variable_labels,  
684   tex = TRUE,  
685   replace = TRUE,  
686   digits = 4,  
687   file = file.path(paths$tables, "decomposition_and_robustness.tex")  
688 )  
689  
690 write_csv(  
691   cumulative_results,  
692   file.path(paths$tables, "cumulative_effects.csv")  
693 )  
694  
695 # Session information records the R version and package environment used to run  
696 # the analysis, improving reproducibility for reviewers.  
697 writeLines(  
698   capture.output(sessionInfo()),  
699   here("output", "session_info.txt")  
700 )  
701  
702
```